CURRICULUM MODULE • BACKEND ENGINEERING

Day 17 — Spring Boot Fundamentals

> **Core Objectives:** Build foundational competency in Spring Boot by bootstrapping a web application, mastering REST controllers, understanding request routing, utilizing constructor-based dependency injection, and configuring runtime properties.

1. Syllabus Overview & Key Concepts

Topic Area	Conceptual Breakdown
Spring Initializr	Project scaffolding tool that configures build systems (Maven/Gradle), Java SDK targets, metadata, and core starters such as `spring-boot-starter-web`.
Project Structure	Standardized Maven convention separating entry point class, domain models, business service components, and web presentation controllers.
Controllers & REST	`@RestController` combines `@Controller` and `@ResponseBody`, serializing Java domain objects directly into JSON/XML payloads.
Request Mappings & HTTP	Routing incoming client HTTP verbs (GET, POST, PUT, DELETE) to designated Java methods using `@GetMapping` and `@PostMapping`.
Dependency Injection (DI)	Inversion of Control (IoC) implemented via constructor injection to supply beans like `@Service` without manual instantiation.
Application Config	Externalized key-value properties defined in `application.properties` injected with `@Value` or structured binders.

2. Project Architecture & File Structure

A standard layered structure ensures separation of concerns, testability, and clear dependency management:

```
day17-demo/
├── pom.xml                                <!-- Maven dependencies (spring-boot-starter-web)
-->
└── src/
    ├── main/
    │   ├── java/com/example/day17demo/
    │   │   ├── Day17DemoApplication.java  <!-- Bootstrap launcher (@SpringBootApplication)
-->
    │   │   ├── model/Student.java         <!-- Domain POJO data entity -->
    │   │   ├── service/StudentService.java<!-- Business logic layer (@Service, DI) -->
    │   │   └── controller/
    │   │       ├── HelloController.java   <!-- Hello World & config-driven endpoint -->
    │   │       └── StudentController.java <!-- REST API handling GET/POST payloads -->
    │   └── resources/
    │       └── application.properties     <!-- Port settings and application properties -->
    └── test/                               <!-- Unit and integration tests -->
```

3. Configuration & Domain Model

Set up application properties in `src/main/resources/application.properties`:

```
server.port=8080
app.greeting.default-message=Welcome to Day 17 Spring Boot Learning!
```

Define the student entity in `src/main/java/com/example/day17demo/model/Student.java`:

```
package com.example.day17demo.model;

public class Student {
    private Long id;
    private String name;
    private String course;
    private double grade;

    public Student() {}
    public Student(Long id, String name, String course, double grade) {
        this.id = id; this.name = name; this.course = course; this.grade = grade;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }
    public double getGrade() { return grade; }
    public void setGrade(double grade) { this.grade = grade; }
}
```

4. Service Layer (Dependency Injection)

Define `StudentService.java` annotated with `@Service` to be managed by the Spring IoC container:

```
package com.example.day17demo.service;

import com.example.day17demo.model.Student;
import org.springframework.stereotype.Service;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Service
public class StudentService {
    private final List<Student> students = new ArrayList<>();

    public StudentService() {
        students.add(new Student(1L, "Alice Johnson", "Computer Science", 3.8));
        students.add(new Student(2L, "Bob Smith", "Information Technology", 3.5));
        students.add(new Student(3L, "Charlie Brown", "Data Science", 3.9));
    }

    public List<Student> getAllStudents() { return students; }

    public Optional<Student> getStudentById(Long id) {
        return students.stream().filter(s -> s.getId().equals(id)).findFirst();
    }

    public Student addStudent(Student student) {
        students.add(student);
        return student;
    }
}
```

5. REST Controllers Implementation

A. Hello World Endpoint: reads configuration via `@Value` in `HelloController.java`:

```
package com.example.day17demo.controller;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class HelloController {

    @Value("${app.greeting.default-message}")
    private String defaultMessage;

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World! " + defaultMessage;
    }
}
```

B. Student Resource Controller: demonstrates constructor injection and HTTP mapping handling:

```
package com.example.day17demo.controller;

import com.example.day17demo.model.Student;
import com.example.day17demo.service.StudentService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {
    private final StudentService studentService;

    // Constructor injection: Spring supplies StudentService automatically
    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @GetMapping
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
        return studentService.getStudentById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND).build());
    }

    @PostMapping
    public ResponseEntity<Student> createStudent(@RequestBody Student student) {
        Student saved = studentService.addStudent(student);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

6. Tasks Checklist & Verification Matrix

Method	Endpoint Path	Input / Payload	Expected Response & Status
`GET`	`/api/hello`	None	`200 OK` — Plain text greeting message.
`GET`	`/api/students`	None	`200 OK` — JSON array containing 3 sample student records.
`GET`	`/api/students/1`	Path Variable: `1`	`200 OK` — JSON record for "Alice Johnson".
`POST`	`/api/students`	JSON: `{"id":4,"name":"Dana",...}`	`201 Created` — Echoes the inserted student object.

7. Expected Output Deliverables

- **Operational Application:** Standalone Spring Boot web service launched via Maven running on port `8080`.
- **Functional Endpoints:** Verified Hello World message and student query endpoints via Web Browser, cURL, or Postman.
- **Clean Architecture:** Separation maintained between presentation controllers, domain models, and service injection.